RoadMap

🚀 PROJECT ROADMAP

1-on-1 Mentor–Student Platform (Industry 9.5–10/10)

🎯 PROJECT GOAL (CLEAR & SAFE STATEMENT)

Build a **web-based 1-on-1 mentorship platform** where a mentor and a student can:

- Join a private session
- Video call in real time
- Chat via messages
- Collaboratively edit code in a shared editor

🧠 TECH STACK (INDUSTRY-APPROVED)

Frontend

- **Next.js (React + TypeScript)**
- **Tailwind CSS**
- **Monaco Editor** (VS Code editor) (You an also use other code editor)
- shadcn/ui (optional)

Backend

- **Node.js**
- **Express.js** (or NestJS if advanced)
- **Socket.io** (real-time sync)
- **WebRTC** (video & audio)

Database

- **PostgreSQL**
- **Supabase** (auth + DB + storage)

Infra & DevOps

- **Vercel** (frontend)
- **Railway / Render** (backend)
- **STUN servers** (free public ones)

🛣️ 2-WEEK ROADMAP (FAST-TRACK MVP)

🔷 WEEK 1 – Foundation + Core Backend + Editor

🎯 Goal:

Get **authentication + sessions + real-time editor working**

🧩 Day 1–2: Planning & Setup

Tasks

- Finalize MVP scope (keep it minimal ⚠️)
- Design architecture (Frontend + Backend + DB + Sockets)
- Create:
  - ER Diagram
  - Basic architecture diagram
- Setup:
  - Next.js + Tailwind
  - Express backend
  - Git repo + branching

Deliverables

- README (initial)
- Architecture + DB design ready

🔐 Day 2–3: Authentication & Roles

Features

- Signup / Login
- Mentor / Student roles
- Protected routes

Tasks

- Setup Supabase Auth
- Store user roles
- Middleware for route protection
- JWT setup for sockets

Output

✅ User can login & role is identified

🧑‍💻 Day 3–5: Session Management

Features

- Create session (Mentor)
- Join session via link (Student)
- Start / End session
- Active session tracking

Tasks

- Design `sessions` table
- Create REST APIs:
    - create session
    - join session
    - end session
- Authorization checks

Output

✅ Mentor creates session → Student joins via link

💻 Day 5–7: Shared Code Editor (CORE 🚀)

Features

- Monaco Editor
- Language selection
- Real-time sync (1-on-1 only)

Tasks

- Integrate Monaco
- Setup Socket.io
- Implement:
  - Code sync
  - Cursor sync (optional)
- Add throttling (important)

Strategy

- Use **last-write-wins**
- Keep it simple (no OT/CRDT)

Output

✅ Real-time collaborative editor working

🔷 WEEK 2 – Communication + Video + Finalization

🎯 Goal:

Add **chat + video + polish + deploy**

💬 Day 8–9: Chat System

Features

- Session-based chat
- Messages with timestamps
- Basic system messages

Tasks

- Socket chat events
- Store messages in DB
- UI chat panel

Output

✅ Working chat inside session

📹 Day 9–11: Video Call (WebRTC)

Features

- 1-on-1 video call
- Mic/camera toggle

Tasks

- Setup WebRTC peer connection
- Exchange:
    - SDP
    - ICE candidates
- Use Socket.io for signaling

⚠️ Scope Control (IMPORTANT)

- ❌ No recording
- ❌ No group calls
- ❌ No screen sharing

Output

✅ Basic video call working

🔗 Day 11–12: Integration

Tasks

- Combine:
  - Editor + Chat + Video
- Handle:
  - Disconnects
  - Reconnect logic
  - Session cleanup

Edge Cases

- User refresh
- User leaves mid-session
- Session timeout

Output

✅ Full flow working smoothly

🚀 Day 13–14: Deployment & Polish

Tasks

- Deploy frontend (Vercel)
- Deploy backend (Railway / Render)
- Setup env variables
- UI polish (loading, errors)

Final Deliverables

- 🌐 Live URL
- 💻 GitHub repo
- 📄 README (important)

- 📽️ Demo video (VERY IMPORTANT)

🎯 FINAL MVP FEATURES (WHAT YOU CAN CLAIM)

Say this confidently in interviews:

> “Built a real-time 1-on-1 mentoring platform with authentication, session-based collaboration, live code editor using sockets, integrated chat system, and WebRTC-based video calling.”

⚡ PRO TIPS (VERY IMPORTANT)

- Focus on **working product > perfect product**
- Avoid over-engineering (no CRDT, no scaling claims)
- Keep UI clean but simple
- Demo video matters more than code sometimes

If you want next step, I can:
✅ Break this into **daily 6-hour schedule (like your previous plans)**
✅ Give **folder structure + APIs + DB schema**
✅ Help you **start Day 1 right now with exact code setup**

-

🧩 DATABASE TABLES (MINIMUM)

- users
- profiles
- sessions
- messages
- code_snapshots (optional)

🧪 BONUS FEATURES (ONLY IF TIME LEFT)

- Session recordings ❌ (advanced)
- Code execution sandbox ⚠️
- AI code suggestions ⚠️

🏆 HOW TO PRESENT THIS PROJECT (IMPORTANT)

Resume Description

> **Built a 1-on-1 mentorship web platform with real-time collaborative code editing, video conferencing using WebRTC, and chat using Socket.io. Designed for mentor–student live coding sessions.**

🚀 Why Industry Likes This Project

🚀 Why Industry Likes This Project

This project demonstrates **real engineering depth**, not just CRUD.

You are showing:

- Real-time systems
- Collaboration features
- WebSockets / WebRTC
- Conflict resolution
- Role-based access
- Performance & sync handling

Very few student projects reach this level.

🔥 Core Features (Perfect Scope)

🔥 Core Features (Perfect Scope)

1️⃣ User Roles

- Mentor
- Student

2️⃣ Session Management

- Schedule 1-on-1 session
- Join via session link
- Time-bound sessions

3️⃣ Shared Code Editor (MOST IMPORTANT)

- Real-time collaborative editing
- Cursor sync
- Language selection (JS / Python)
- Run code (optional)

4️⃣ Video Conferencing

- 1-on-1 video call
- Screen sharing (optional)
- Mute / camera controls

5️⃣ Messaging / Chat

- Session-based chat

- Code snippets
- System messages

📈 How Reviewers Will Judge This

📈 How Reviewers Will Judge This

Area	Verdict
Complexity	⭐⭐⭐⭐⭐
Real-world relevance	⭐⭐⭐⭐⭐
Resume impact	⭐⭐⭐⭐⭐
Achievability	⭐⭐⭐⭐☆
Uniqueness	⭐⭐⭐⭐☆

🧠 Final Honest Verdict

✅ This project is:

- **NOT unrealistic**
- **NOT over-ambitious**
- **EXTREMELY impressive if done well**

🏆 Rating Potential

- **9.5/10** (normal execution)
- **10/10** (polished + deployed + demo video)

🔥 Want to Level This Up to Guaranteed 10/10?

I can:

- Design **system architecture diagram**

- Provide **WebSocket sync logic**
- Give **Monaco Editor integration code**
- Provide **WebRTC 1-on-1 setup**
- Create **resume-safe project description**

🎯 FINAL INDUSTRY VERDICT

Factor	Rating
Complexity	⭐⭐⭐⭐⭐
Real-world relevance	⭐⭐⭐⭐⭐
Resume impact	⭐⭐⭐⭐⭐
Achievable by students	⭐⭐⭐⭐☆

👉 **Overall: 9.5–10 / 10**

Resource List

🔹 1. NEXT.JS (CORE FRONTEND)

📺 YouTube (BEST 🔥)

Search:
👉 “Next.js full course freeCodeCamp”

👉 Covers:

- Routing
- Pages / App Router
- API calls
- SSR vs CSR

📚 Documentation

- Next.js Docs
  👉 https://nextjs.org/docs

💡 Learn:

- App Router
- Layouts
- Client vs Server components

🔹 2. REACT FUNDAMENTALS (IF NEEDED)

📺 YouTube

Search:
👉 “React full course 2024”

📚 Docs

- React Docs
  👉 https://react.dev/

👉 Focus only on:

- Hooks (useState, useEffect)
- Component structure

🔹 3. TAILWIND CSS (UI FAST BUILDING)

📺 YouTube

Search:
👉 “Tailwind CSS crash course”

📚 Docs

- Tailwind CSS Docs
  👉 https://tailwindcss.com/docs

👉 Learn:

- Flexbox
- Grid
- Responsive design

🔹 4. MONACO EDITOR (CODE EDITOR 🔥)

📺 YouTube

Search:
👉 “Monaco editor React tutorial”

📚 Docs

- Monaco Editor Docs
  👉 https://microsoft.github.io/monaco-editor/

👉 Learn:

- Editor setup
- onChange events
- Language switching

💡 Important

This is your **CORE FEATURE**

- Capture code changes
- Send via WebSocket

🔹 5. WEBSOCKETS (REAL-TIME)

📺 YouTube

Search:
👉 “WebSocket JavaScript tutorial”

📚 Docs

- WebSocket API Docs
  👉 https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

👉 Learn:

- connect
- send
- receive messages

🧠 Use in your project

- Code sync
- Chat
- WebRTC signaling

🔹 6. SOCKET.IO (OPTIONAL)

📺 YouTube

Search:
👉 “Socket.io client tutorial”

📚 Docs

- Socket.IO Docs
  👉 https://socket.io/docs/v4/

👉 Only if:

- You choose Socket.io instead of raw WebSockets

🔹 7. CHAT UI + STATE MANAGEMENT

📺 YouTube

Search:
👉 “React chat UI tutorial”

💡 Learn:

- Message list rendering
- Auto scroll
- Input handling

🔹 8. WEBRTC (VIDEO CALL 🔥🔥)

📺 YouTube (VERY IMPORTANT)

Search:
👉 “WebRTC crash course”

📚 Docs

- WebRTC Docs
  👉 https://webrtc.org/getting-started/overview

👉 Learn:

- getUserMedia
- RTCPeerConnection
- SDP & ICE candidates

🧠 Key Insight

- Frontend handles:
  - Video stream
  - Peer connection
- Backend only signals

🔹 9. API INTEGRATION (FRONTEND ↔ BACKEND)

📺 YouTube

Search:
👉 “Fetch API React tutorial”
👉 “Axios React tutorial”

📚 Docs

- Axios Docs
  👉 https://axios-http.com/docs/intro

👉 Learn:

- GET / POST requests
- Auth headers

🔹 10. AUTH HANDLING (FRONTEND)

📺 YouTube

Search:
👉 “JWT authentication React”

📚 Docs

- Supabase Auth Docs
  👉 https://supabase.com/docs/guides/auth

👉 Learn:

- Login/signup
- Token storage
- Protected routes

🔷 11. UI COMPONENTS (OPTIONAL BUT FAST)

📚 Docs

- shadcn/ui
  👉 https://ui.shadcn.com/

👉 Use for:

- Buttons
- Modals
- Forms

🔷 12. DEPLOYMENT

📺 YouTube

Search:
👉 “Deploy Next.js Vercel”

📚 Docs

- Vercel Docs
  👉 https://vercel.com/docs

🧠 FINAL FRONTEND LEARNING PATH

Follow this order:

1. Next.js basics
2. Tailwind CSS
3. API integration
4. Monaco Editor
5. WebSockets
6. Chat UI
7. WebRTC
8. Auth handling
9. Deployment